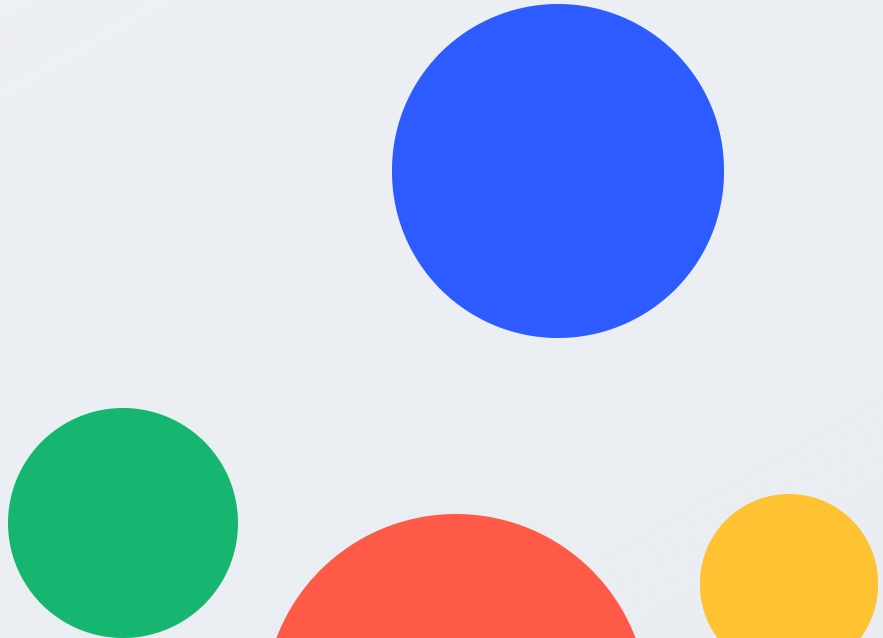
CHAPTER 7 · YEAR-1 CORE · UNIT TESTING

It runs. But is it *right*?

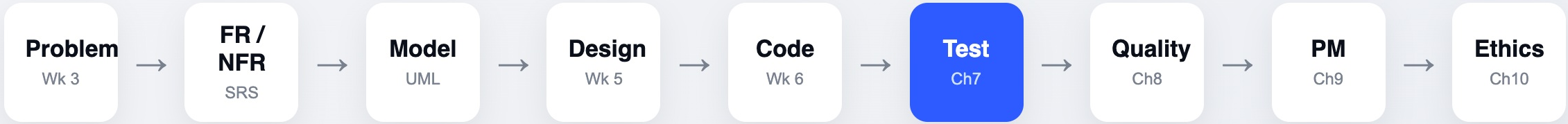
A program that starts up is not a program that works. **Testing** is how an engineer proves the code does what the requirement asked — and how you catch what **AI-generated code** quietly got wrong, before a user does.



▶ **intro2se-ch7-testing.fig** · Where we are

THE GOLDEN THREAD · ONE STEP FURTHER

Testing closes the loop from requirement to running code



IDEA 1

Why test?

Verification vs validation — “built it right” vs “built the right thing.”

IDEA 2 · 3

Unit tests

A test = input + expected output. A unit test checks one function in isolation.

IDEA 4 · 5

AC → tests

Each acceptance criterion becomes a test — and tests are how you catch AI mistakes.

Scope reminder: testing is **understand-only** this term — you learn the concepts and apply them to harden your prototype; there is no separately graded test suite.

IN PLAIN TERMS

A criterion and a test are the **same sentence written twice** — once in plain English (“a done task is not reminded”), once in code (the `assert`). Writing the test is just translating that sentence into something the computer can read.

IDEA 1 · WHY WE TEST

Two questions every test answers

VERIFICATION

Did we build it *right*?

Does the code match the spec — no bugs, correct logic? Caught by unit tests & reviews.

VALIDATION

Did we build the *right thing*?

Is this what the user actually needed? Caught by demos, acceptance tests, feedback.

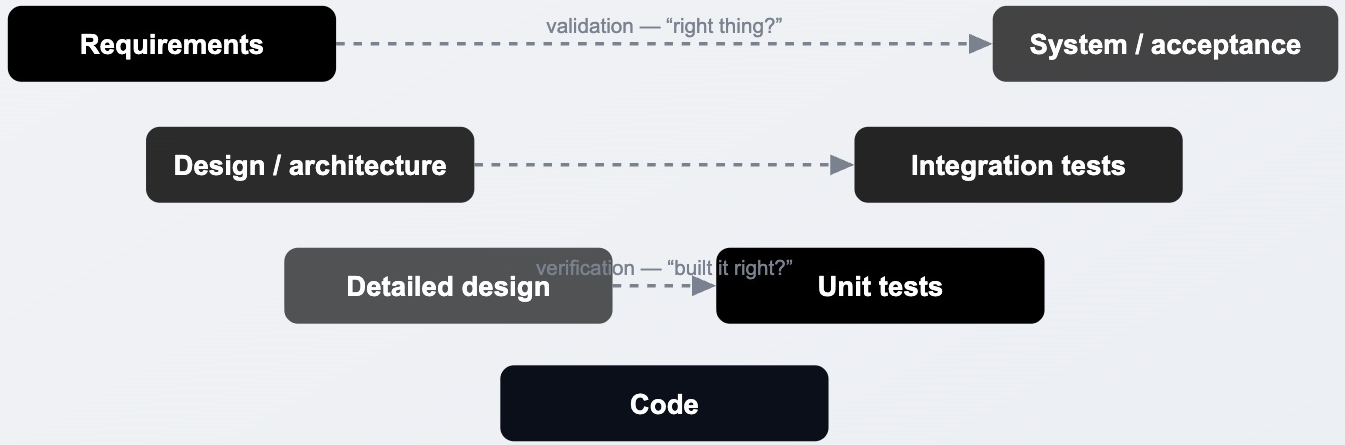
IN PLAIN TERMS

A reminder that fires on the **wrong day** still “runs” — it passes a glance but fails *verification*. A perfectly-coded feature nobody asked for passes verification but fails *validation*. You need both.

▶ **intro2se-ch7-testing.fig** · The V-model

IDEA 1B · THE V-MODEL (SOMMERVILLE §8)

Every build step has a matching check



Left arm = what you specify · right arm = how you check it · each level tests the one across from it

V&V framing from **Boehm (1979)**: *Validation* = are we building the right product? *Verification* = are we building the product right? (Sommerville 9e, §8).

IN PLAIN TERMS

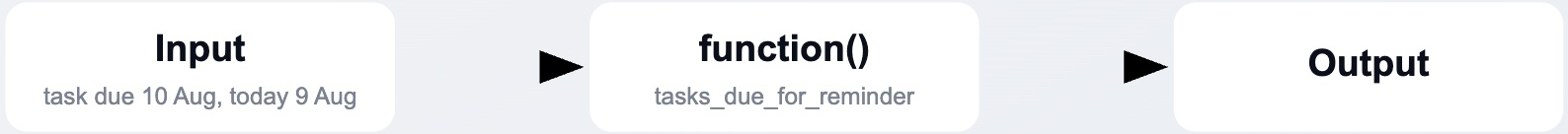
It's **writing** (left arm) and **proofreading** (right arm). You proofread each part against what you meant to write: verification = “no typos, matches the plan”; validation = “this is the chapter the reader actually needed.”

▶ **intro2se-ch7-testing.fig** · A test = input + expected

IDEA 2 · WHAT A TEST ACTUALLY IS

Give known input → check the result equals what you expect

If the real output matches the expected output, the test **passes**. If not, it **fails** — and you just found a bug, cheaply, before a user did.



Output = Expected ? **✓ pass / ✗ fail = bug**

A test is a tiny, repeatable experiment on your code

IN PLAIN TERMS

Type 2 + 2 into a calculator and you **expect** 4. If the screen shows 5, the calculator has a bug. A test is exactly that check — an expected answer you write down once, so the computer re-checks it for you every time.

IDEA 3 · THE SMALLEST USEFUL TEST

A unit test checks one small piece in isolation

ONE UNIT

Usually one function

Test `tasks_due_for_reminder()` alone — no database, no UI, no network.

FAST

Runs in milliseconds

So you run them constantly — after every change, not once at the end.

PRECISE

Pinpoints the break

When one fails, you know *exactly* which piece broke — no hunting.

IN PLAIN TERMS

Unit tests are like checking each **LEGO brick** is the right shape before you build the castle. Cheaper to catch a bad brick now than to tear down the whole thing later.

IDEA 4 · THE GOLDEN THREAD REACHES TESTING

Each acceptance criterion becomes a test

FROM YOUR SRS

- **FR-1** — remind an open task 1 day before it is due
- AC: open task due tomorrow → *appears* in reminders
- AC: a `done` task → does *not* appear

Problem → FR → Design → Code → **Test**. Same thread, one step further.

Two criteria, two tests

```
def test_reminds_open_task_1_day_before():
  tasks = [Task(due=date(2026,8,10), status='open')]
  assert tasks_due_for_reminder(tasks, date(2026,8,9)) == tasks

def test_does_not_remind_done_task():
  tasks = [Task(due=date(2026,8,10), status='done')]
  assert tasks_due_for_reminder(tasks, date(2026,8,9)) == []
```

CHOOSING TEST INPUTS · TWO CLASSIC TECHNIQUES (SWEBOK · SOMMERVILLE §8)

You can’t test every input — so test the right few

EQUIVALENCE PARTITIONING (EP)

Group inputs that behave the same

Split inputs into **classes** the code treats alike, then test **one value from each** — one valid, one too short, one too long. One test now stands for a whole range.

BOUNDARY VALUE ANALYSIS (BVA)

Bugs hide at the edges

Test right **at and around each boundary** — the first and last allowed value, and just outside. This is where “≤ vs <” and off-by-one bugs live.



StudyMate: a task title must be 1–100 chars — EP picks one per band (0 · 50 · 101); BVA adds the edges (1 · 100)

Five well-chosen tests beat a hundred random ones. Same idea for a **due date** (past / today / future) or a reminder window. Source: Sommerville 9e §8.1.2 (choosing test cases — partitions & boundaries); SWEBOK v4 Testing KA.

▶ **intro2se-ch7-testing.fig** · Levels of testing

IDEA 3B · LEVELS OF TESTING (SOMMERVILLE §8.1–§8.4)

Many small tests at the base, a few big ones on top



1 **Development testing (§8.1)**
You, during the build: **unit** → component → system tests.

2 **Release testing (§8.3)**
A separate check of the whole version before users get it.

3 **User testing (§8.4)**
Real users try it — **acceptance testing** is the formal form.

Pyramid shape: Cohn (2009). Most of your effort is cheap unit tests at the base.

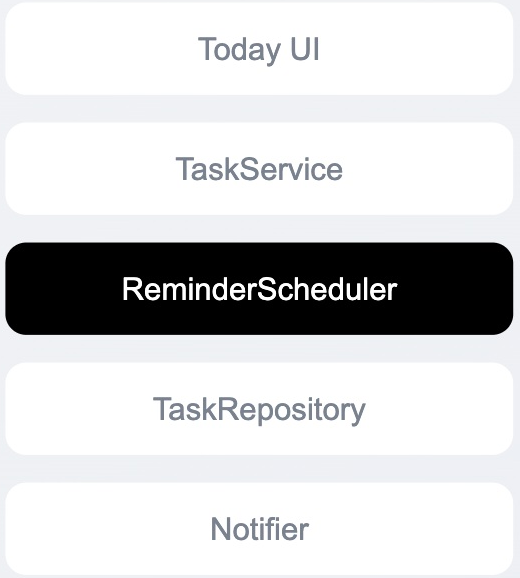
IN PLAIN TERMS

Like checking a car: test each part on the bench many times (cheap, fast = **unit tests**), then take a few full test-drives (slow, expensive = **system tests**). Most of your checking lives at the bottom.

LEVELS OF TESTING · 1/5 · UNIT

Unit testing — one function, in isolation

Testing individual program components — methods or object classes — in isolation (Sommerville §8.1.1). A defect-testing process: fast (milliseconds), and a failure points at exactly one piece.



highlighted = tested at this level

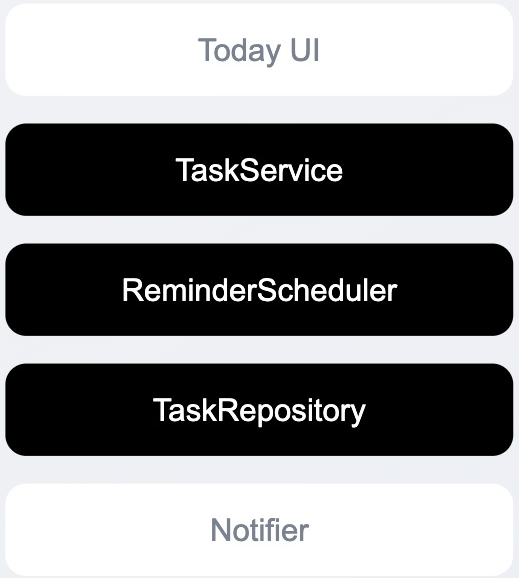
1. **`days_until_due(task, today)`**
   Due 10 Aug, today 8 Aug → returns `2`. · core date maths, zero dependencies
2. **Boundaries of that function**
   Overdue → a *negative* number; due *today* → `0`. · off-by-one is the classic bug
3. **`tasks_due_for_reminder(…)`**
   An `open` task due tomorrow → is *in* the list. · the happy path of FR-1
4. **The “should-not” case**
   A `done` task → is *not* reminded. · proves the filter really filters
5. **`is_valid_title(title)`**
   Empty / spaces-only → rejected; a normal title → accepted. · input validation in isolation

Base of the pyramid — the most numerous, fastest, cheapest tests. Mostly **verification**.

LEVELS OF TESTING · 2/5 · INTEGRATION

Integration / component testing — modules via their interfaces

Several units combined into composite components, tested through their interfaces (Sommerville §8.1.3; SWEBOK calls it *integration testing*). The bugs live *between* the pieces, not inside one unit.



highlighted = tested at this level

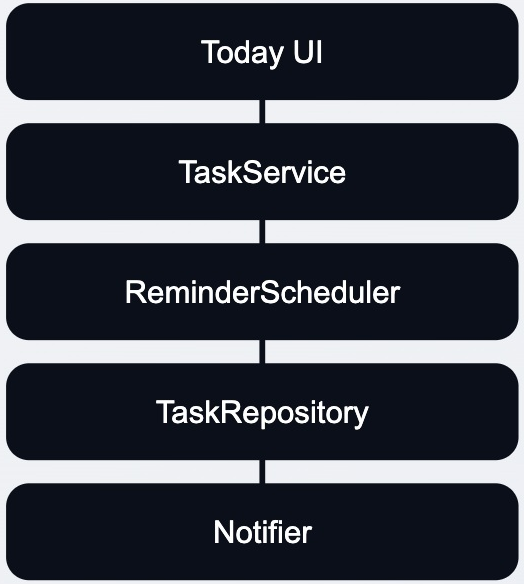
The other level you'll actually build on your prototype in this course.

1. **Add via the service**
   `TaskService.add(…)` → stored in `TaskRepository` and read back. · logic + persistence together
2. **Scheduler + repo + notifier**
   A due task flows through and a reminder is created by the notifier. · three modules, one path
3. **Mark done, end of chain**
   Stored status updates *and* the task drops from tomorrow's reminders. · one action, two modules react
4. **Invalid data at the boundary**
   A bad due date is rejected; *nothing* is written to the repository. · validation guards persistence
5. **Today screen ↔ repository**
   The component queries the repo and renders the correct list for today. · UI reads real data correctly

LEVELS OF TESTING · 3/5 · SYSTEM

System testing — the whole app, end-to-end

Integrate the components into a version of the whole system and test it together — that they interact correctly and pass the right data across interfaces (Sommerville §8.1.4). Also covers performance & security.



highlighted = tested at this level

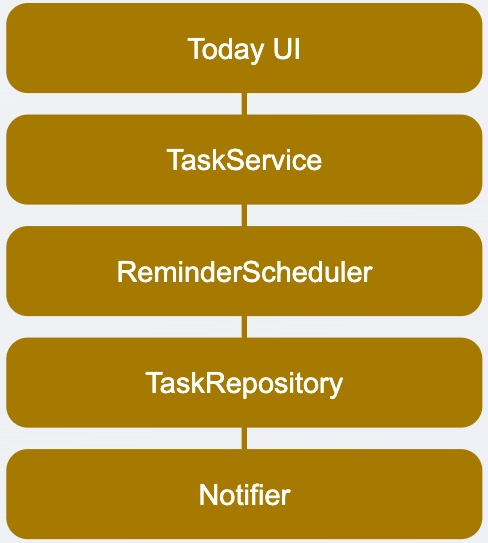
Understand-only in this course — know what it is and why it matters.

1. **Add → see it**
   Add a task with a due date, save → it appears on the Today screen. · a full feature, end to end
2. **Reminder fires**
   A task due tomorrow → a notification at the scheduled time. · FR-1, across the real app
3. **Done disappears**
   Removes it from the active list *and* reminders; counts it complete. · state consistent everywhere
4. **Access control**
   User A cannot see or open user B's tasks. · security, system-wide (Ch8)
5. **Performance (NFR)**
   With 500 tasks, the Today screen loads in < 2 s. · measure it, don't assume it

LEVELS OF TESTING · 4/5 · RELEASE

Release testing — a separate team, the full version

A separate team tests a complete version meant for use outside the dev team, checking it meets stakeholders' requirements (Sommerville §8.3). A form of system testing, done before shipping.



✓ v1.0 — ready to ship

highlighted = tested at this level

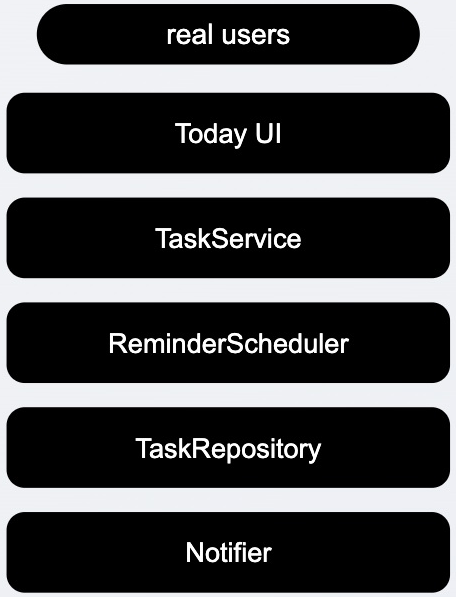
1. **Requirements coverage**
   Every FR (FR-1 remind, FR-2 list, FR-3 edit…) has a passing end-to-end check. · nothing promised is missing
2. **Target platforms**
   Installs and runs on supported devices (e.g. iOS + Android). · works where users are
3. **No regression after update**
   An existing user's saved tasks still load on the new version. · new code didn't break old data
4. **Realistic failure**
   No internet → a clear message, no crash, the reminder is queued. · graceful under bad conditions
5. **Data safety**
   No secrets or real personal data leak into logs or responses. · ship-ready and PDPA-aware (Ch8)

Understand-only — done by a separate team before shipping.

LEVELS OF TESTING · 5/5 · ACCEPTANCE

Acceptance / user testing — real users validate

Users test it in their own environment; in acceptance testing the customer formally decides whether to accept it (Sommerville §8.4). This is *validation* — you can pass every level below and still fail here.



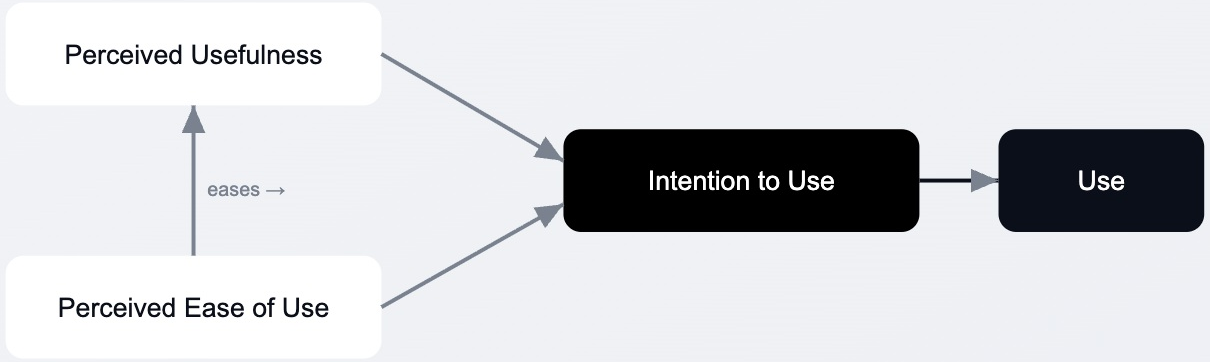
highlighted = tested at this level

1. **Real use**
   A student uses StudyMate for a week → the reminders really help. · does it solve the problem?
2. **Formal sign-off**
   The customer runs the agreed SRS acceptance scenarios and accepts. · contractual validation
3. **Usability**
   A first-timer adds a task and sets a reminder, no instructions, in < 1 min. · learnable without a manual
4. **Right timing**
   The reminder time (1 day before, a useful hour) matches what users want. · validated, not assumed
5. **Fits real habits**
   Users confirm the done / Today flow matches how they really plan study. · the right mental model

The only level that proves **validation** — the right thing, not just built right. How to measure it → next: TAM / UTAUT2 / SUS.

ACCEPTANCE · TECHNOLOGY ACCEPTANCE MODEL (DAVIS, 1989)

TAM: two things decide whether people adopt it



Usefulness + ease of use → intention → use

EXAMPLE · STUDYMATE SURVEY (RATE 1–5)

PU: “StudyMate helps me not miss deadlines.”

PEOU: “Adding a task is easy.” → average each.

WHEN TO USE

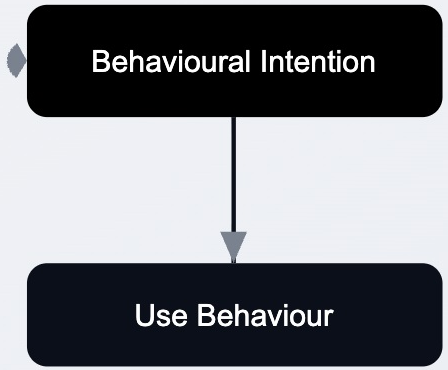
Early, quick read on *why* people would adopt a **work / productivity** tool — only two levers, a short survey, easy to analyse.

Source: Davis, F. D. (1989), *MIS Quarterly* 13(3). Ease of use also raises usefulness (easier → feels more useful).

ACCEPTANCE · UTAUT2 (VENKATESH, THONG & XU, 2012)

UTAUT2: seven drivers of whether people use it

- **PE** Performance Expectancy
- **EE** Effort Expectancy
- **SI** Social Influence
- **FC** Facilitating Conditions
- **HM** Hedonic Motivation
- **PV** Price Value
- **HB** Habit



moderated by: age · gender · experience

`seven drivers → intention → use`

EXAMPLE · STUDYMATE (RATE 1–5)

“Helps my grades” (PE) · “easy to learn” (EE) · “my friends use it” (SI) · “I open it automatically” (Habit).

WHEN TO USE

When you need a **fuller, consumer-app** picture — adds motivation, price, habit & social factors beyond TAM. Heavier survey; usually in research / theses.

Source: Venkatesh, Thong & Xu (2012), *MIS Quarterly* 36(1); extends UTAUT (Venkatesh et al., 2003).

ACCEPTANCE · SYSTEM USABILITY SCALE (BROOKE, 1996)

SUS: ten questions → one score out of 100

1 Ask 10 statements
Alternating positive / negative, each rated 1–5 (strongly disagree → agree).

2 Score each item
Odd (positive): response − 1 · Even (negative): 5 − response.

3 Sum × 2.5
Gives one number from **0 to 100** — it is *not* a percentage.

4 Interpret
~68 = average · 80+ = excellent · under 50 = poor.

Example SUS items (StudyMate)

+	“I would use StudyMate often.”
−	“StudyMate was unnecessarily complex.”
+	“StudyMate was easy to use.”

WHEN TO USE SUS
Short, free, battle-tested — run it on **any** prototype. 5–10 users after a week → a number you compare across versions and put in your report.



A SUS score is interpreted on this 0–100 scale — aim above the ~68 average (Bangor et al., 2009)

Sources: Brooke (1996) “SUS: a quick and dirty usability scale”; Bangor, Kortum & Miller (2009) for grades / adjective ratings.

ACCEPTANCE · HOW TO RUN A USABILITY TEST — SUS COMES LAST

Users do real tasks first — then fill the SUS

The order of a session

1. **Write task scenarios**
   Realistic goals a user should be able to accomplish.
2. **Recruit 5–10 target users**
   Here: students who'd actually use StudyMate.
3. **Brief — "think aloud"**
   They narrate; **you don't help or hint**.
4. **They do each task — you observe**
   Note success / time / errors / where they get stuck.
5. **Then fill the SUS**
   Right after the tasks — 10 items, rated 1–5.
6. **Score + fix + retest**
   SUS 0–100 with your notes → fix worst spots, repeat.

Example task scenarios (StudyMate)

- **T1 Add a task due Friday**
  Create "Submit report" with this Friday's date. · done without help?
- **T2 Turn on a reminder**
  Set it to remind 1 day before due. · finds the setting?
- **T3 Mark a task done**
  Complete an existing task. · obvious control?
- **T4 Find overdue tasks**
  Show which tasks are past due. · can they tell?
- **T5 Change a due date**
  Move a task to next week. · edit discoverable?

WHY THIS ORDER

SUS measures the *feeling* of usability; the **task observations tell you where it broke**. You need both — and SUS is only meaningful **after** people have actually used it on real tasks.

Observe during the tasks; administer SUS immediately after. Source: Brooke (1996); standard usability-testing practice (e.g. Nielsen, 5-user testing).

ACCEPTANCE · THE 10 SUS ITEMS, WRITTEN FOR STUDYMATE · RATED 1–5

The SUS questionnaire for StudyMate

Positive items (odd) — score = answer − 1

1	I would like to use **StudyMate** frequently.
3	I thought StudyMate was **easy to use**.
5	The functions in StudyMate were **well integrated**.
7	Most people would **learn StudyMate very quickly**.
9	I felt **very confident** using StudyMate.

Negative items (even) — score = 5 − answer

2	StudyMate was **unnecessarily complex**.
4	I'd need **technical support** to use StudyMate.
6	There was **too much inconsistency** in StudyMate.
8	StudyMate was **very cumbersome** to use.
10	I needed to **learn a lot** before I could get going.

HOW TO USE THESE

Each rated **1 (strongly disagree) – 5 (strongly agree)**, filled **right after the tasks**. Convert (odd: −1 · even: 5−), sum, × 2.5 → a 0–100 score.

The 10 standard items (Brooke, 1996) with "the system" replaced by "StudyMate." Keep the wording & the alternating +/− pattern.

▶ **intro2se-ch7-testing.fig** · Good test vs bad test

WRITING TESTS THAT ARE WORTH KEEPING

A good test is small, clear, and about one thing

DO

Name it like a sentence

`test_does_not_remind_done_task` — the name says what should happen.

Test the boundaries

Empty list, the exact due date, a huge list — the edges are where bugs hide.

One reason to fail

Each test checks one behaviour, so a failure points at one cause.

DON'T

Test only the happy path

“It works when everything is perfect” proves almost nothing.

Assert twenty things at once

When it fails you won't know which part broke.

Depend on today's date / the network

A test that passes Monday and fails Tuesday isn't a test — it's a coin flip.

IN PLAIN TERMS

A good test is like a good exam question: it asks about **one** thing, and a wrong answer tells you exactly what the student — here, your code — got wrong.

AI-FIRST · TESTS ARE HOW YOU CATCH THE AI

AI writes the happy path; your tests find the edges

AI OFTEN MISSES…	…AND THE TEST THAT CATCHES IT
The empty case	No tasks at all → should return `[]`, not crash.
The "done" / excluded case	A completed task must *not* be reminded.
The exact boundary	Due *today* vs due *tomorrow* — off-by-one is the classic AI bug.
Silent wrong logic	Runs fine, returns the wrong list — only an assertion reveals it.

THE RULE

Ask AI for the code *and* the tests — then **check the tests cover your acceptance criteria**. It almost always leaves one out. Finding that gap is the engineering, and it's what you're graded on.

CASE STUDY · WHEN A REUSED UNIT WENT UNTESTED

Ariane 5, Flight 501 — a 37-second, ~$370M bug

WHAT HAPPENED

Reused code, new rocket

Inertial-reference software was **reused from Ariane 4**. Ariane 5 flew faster, so a horizontal-velocity value overflowed when a 64-bit number was converted into a 16-bit one. The unhandled error shut down **both** the primary and backup units; the rocket veered and self-destructed 37 s after liftoff.

WHY A TEST WOULD HAVE CAUGHT IT

The new range was never tested

The conversion was **never tested with Ariane 5's larger velocity values** — the reused path had no test for the new operating range, and the code assumed the value could not overflow.

THE LESSON — STRAIGHT BACK TO THIS CHAPTER

Reused code is not tested code. Re-test every reused component against the *new* context's inputs and boundaries — exactly the edge cases (Idea 5) that AI-generated code also skips.

Source: ESA/CNES Ariane 501 Inquiry Board Report (Lions et al., 1996).

TEAM STUDIO · ~45 MIN

Turn two acceptance criteria into two passing tests

STEP A · 10 MIN

Pick the logic

Choose one function at the heart of an FR — the piece that decides something.

STEP B · 25 MIN

Write 2–3 tests

One happy path, one edge, one “should-not”. Prompt AI, then **read & fix** them.

STEP C · 10 MIN

Run until green

Make a test fail on purpose first — prove it actually checks something.

Leave with at least one test that **failed, then passed** after you fixed the code. That is the whole loop in miniature.

DO THIS WITH YOUR TEAM

Use tests to harden your prototype

PROJECT STEP

Protect the core logic

Write tests for the one or two functions your demo absolutely cannot get wrong.

Trace to the SRS

Point each test at an acceptance criterion — that's your evidence at the oral exam.

AI-FIRST — VERIFY!

Ask AI for test cases

Then check they truly cover the edges — add the one it forgot.

Never ship a red bar

If a test is failing, the feature is not done. Fix the code, not the test.

🧠 AI CRITIQUE #7 · LAST 10 MIN · ON A4

Spot it · explain it · fix it

> “I added a test so you're covered: `assert add_task('Buy milk') is not None`. Green across the board — ship it.”
>
> *— an AI coding assistant, confidently*

1 · SPOT

What's weak?

The test only checks “something came back.” It never checks the task was stored correctly, with the right title or due date.

2 · EXPLAIN

Why is it wrong?

A test that can't fail for the real bug gives false confidence. “Not None” passes even if the data is garbage.

3 · FIX

The right way

Assert on the *result*: the stored task's title equals `'Buy milk'`, status is `open`, and an invalid input is rejected.

Hand in your A4 before you leave — this is your attendance + AI-critique mark for the week.

REFERENCES · WHERE THIS COMES FROM

Textbook & source references

1. **Sommerville, I. (2011) Software Engineering, 9e**
   Ch8 Software Testing — §8.1 Development testing, §8.2 TDD, §8.3 Release testing, §8.4 User testing.

2. **SWEBOK v4 (IEEE, 2024)**
   Software Testing KA — test levels, techniques, test-driven development.

3. **Boehm, B. (1979)**
   Verification vs validation — “build it right” vs “build the right thing.”

4. **Cohn, M. (2009) Succeeding with Agile**
   The test pyramid — many unit tests, few end-to-end tests.

5. **ESA/CNES (1996) Ariane 501 Inquiry Board Report**
   The reused-and-untested overflow that destroyed Flight 501.

6. **Acceptance & usability measurement**
   Davis (1989) TAM; Venkatesh et al. (2003, 2012) UTAUT / UTAUT2; Brooke (1996) & Bangor et al. (2009) SUS.

Grounded in the course's own Sommerville 9e + SWEBOK v4 (in `raw/`). Read §8 before the exam; SUS is the acceptance measure you can actually run.

CHAPTER 7 · IN FOUR LINES

Running isn't working — prove it with tests

WHY

Verification + validation

Built it right, and built the right thing — tests and demos cover both.

WHAT

Input → expected output

A unit test checks one function in isolation — fast, precise, repeatable.

THREAD

Acceptance criteria become tests

Every AC is a test — the Golden Thread reaching into your code.

AI-FIRST

Tests catch what AI missed

Generate, then verify coverage. The gap it leaves is yours to find.

Reading: Sommerville Ch8 · Valente (Software Testing) · SWEBOK: Software Testing. Next: Ch8 — Software Quality & Basic Security.